
Effortless Speed Controller (ESC): Mengatur Kipas Angin Semudah Mengangkat Jari

Dalam era di mana teknologi terus berkembang pesat, kenyamanan dan efisiensi menjadi aspek yang paling dicari. **Effortless Speed Controller (ESC)** hadir sebagai solusi revolusioner untuk mengatur kecepatan kipas angin hanya dengan **gesture jari tangan kiri**. Mengapa tangan kiri? Karena tangan kiri sering kali kurang dimanfaatkan dalam aktivitas sehari-hari, membuat teknologi ini menjadi **intuitif, efisien, dan inovatif**.

Konsep Inovasi

ESC menggabungkan **teknologi pengenalan gesture** dan **kontrol kecepatan motor** pada kipas angin. Alat ini bekerja dengan mendeteksi jumlah jari yang ditunjukkan oleh **tangan kiri** pengguna, di mana setiap gesture jari memiliki arti khusus untuk mengontrol kecepatan kipas:

- **1 Jari** : Very Low – Hembusan angin paling lembut, ideal untuk relaksasi.
- **2 Jari** : Low – Hembusan angin ringan, cocok untuk suasana tenang.
- **3 Jari** : Medium – Kecepatan sedang, pas untuk suasana santai.
- **4 Jari** : High – Angin lebih kuat, membantu pendinginan cepat.
- **5 Jari** : Very High – Angin maksimal, ideal untuk suasana panas.
- **Tangan Mengepal** : Stop – Kipas mati seketika hanya dengan mengepalkan tangan.

Dengan hanya **mengangkat tangan kiri** dan menunjukkan gesture jari, pengguna dapat mengontrol kipas angin tanpa menyentuh tombol atau remote, bahkan saat sibuk atau berada di jarak tertentu.

Bagaimana ESC Bekerja?

1. Deteksi Gesture Tangan:

ESC menggunakan **kamera atau sensor pendeteksi tangan** untuk membaca gesture jari tangan kiri. Algoritma cerdas mendeteksi jumlah jari yang diangkat dengan presisi tinggi.

2. Pengaturan Kecepatan Motor:

Data gesture yang diterima dikirim ke **mikrokontroler ESP32**, yang kemudian mengatur **PWM (Pulse Width Modulation)** untuk mengontrol kecepatan kipas angin secara dinamis.

3. Feedback Real-Time:

Informasi tentang kecepatan kipas ditampilkan pada layar **LCD I2C**, sehingga pengguna dapat melihat mode kecepatan yang sedang aktif.

4. Simpanan Data:

ESC dilengkapi **EEPROM** yang menyimpan pengaturan terakhir kecepatan kipas, sehingga setelah dinyalakan kembali, kipas akan memulai dengan kecepatan yang sama seperti sebelumnya.

Keunggulan ESC (Effortless Speed Controller)

1. Kontrol Tanpa Sentuhan:

Tidak perlu remote atau saklar. Cukup angkat tangan kiri, gesture jari Anda adalah kendali Anda.

2. Praktis dan Efisien:

Membebaskan tangan kanan untuk tetap produktif, sementara tangan kiri menjadi pengontrol yang praktis.

3. Teknologi Inklusif:

ESC cocok digunakan oleh siapa saja, termasuk mereka yang memiliki keterbatasan mobilitas untuk menekan tombol atau mencari remote.

4. Responsif dan Cepat:

Dengan **delay minimal**, ESC merespons gesture tangan dalam hitungan milidetik untuk memastikan pengalaman pengguna yang mulus.

5. Hemat Energi:

Kipas mati secara otomatis ketika tangan mengepal, mencegah konsumsi daya berlebih.

Manfaat Penggunaan ESC

- **Dalam Pekerjaan:** Bayangkan Anda sedang mengetik, membaca, atau melakukan aktivitas lainnya. Cukup angkat tangan kiri, tanpa berpindah posisi, kipas langsung menyesuaikan kecepatan.
- **Dalam Situasi Sibuk:** ESC sangat cocok untuk multitasking. Tidak perlu repot-repot mencari remote atau bergerak ke saklar.
- **Dalam Kehidupan Sehari-Hari:** Bagi orang tua atau penyandang disabilitas, kontrol gesture ini memberikan **kemudahan** dan **aksesibilitas** dalam mengatur kipas angin.

Penutup: Teknologi untuk Kehidupan yang Lebih Mudah

Effortless Speed Controller (ESC) bukan sekadar alat, melainkan solusi inovatif yang mengubah cara kita berinteraksi dengan peralatan sehari-hari. Dengan teknologi gesture tangan kiri yang intuitif, **ESC menghadirkan kenyamanan, kepraktisan, dan efisiensi** hanya dengan gerakan sederhana.

Kini, mengatur kecepatan kipas angin menjadi semudah **mengangkat tangan kiri**. Karena terkadang, **kemajuan teknologi** datang dari hal-hal sederhana yang membuat hidup kita jauh lebih mudah.

LAMPIRAN

Program dengan bahasa python pada PC(mini PC, Raspberry PI, Laptop):

```
import cv2          # Library untuk pemrosesan citra/video
import time         # Library untuk pengaturan waktu
import os           # Library untuk operasi file dan folder
import HandTrackingModule as htm # Modul khusus untuk deteksi tangan
import serial        # Library untuk komunikasi serial (dengan hardware)

# Mengatur port serial untuk komunikasi dengan hardware(ESP32)
ser = serial.Serial('COM22', 115200) # Port 'COM22' dan baudrate 115200
bps
time.sleep(2) # Tunggu 2 detik agar koneksi serial stabil

# Mengatur resolusi video (lebar dan tinggi)
wCam, hCam = 640, 480

# Mengaktifkan kamera (index 1 untuk kamera kedua, bisa disesuaikan)
cap = cv2.VideoCapture(1)
cap.open(0, cv2.CAP_DSHOW) # Membuka video capture menggunakan
DirectShow (untuk Windows)
cap.set(3, wCam) # Set lebar frame
cap.set(4, hCam) # Set tinggi frame

# Membaca gambar overlay dari folder untuk menampilkan jumlah jari
folderPath = r'C:\belajar coding\hand\FingerImages' # Path ke folder
berisi gambar
myList = os.listdir(folderPath) # Membaca semua file di folder
print(myList) # Cetak daftar file gambar yang ditemukan

# Menginisialisasi list untuk menyimpan gambar overlay
overlayList = []
for imPath in myList:
    image = cv2.imread(f'{folderPath}/{imPath}') # Membaca gambar dari
file
    overlayList.append(image) # Menambahkan gambar ke dalam list

print(len(overlayList)) # Cetak jumlah gambar overlay yang dimuat
pTime = 0 # Inisialisasi waktu sebelumnya (untuk menghitung FPS)

# Inisialisasi objek hand detector dengan tingkat kepercayaan
(detectionCon) = 1
detector = htm.handDetector(detectionCon=1)

# Index titik ujung jari berdasarkan model deteksi tangan
tipIds = [4, 8, 12, 16, 20] # ID untuk ujung ibu jari, telunjuk, jari
tengah, manis, dan kelingking

# Variabel untuk mengatur penundaan pengiriman data serial
last_sent_time = time.time() # Waktu terakhir data dikirim
delay_time = 0.1 # Delay waktu pengiriman data (100ms)
tempSend = 6 # Variabel untuk menghindari pengiriman data berulang
```

```

# Loop utama program
while True:
    success, img = cap.read(1) # Membaca frame dari kamera
    img = cv2.flip(img, 1) # Membalik gambar secara horizontal (mirror)

    img = detector.findHands(img) # Deteksi tangan pada frame
    lmList = detector.findPosition(img, draw=False) # Ambil posisi
titik-titik tangan

    if len(lmList) != 0: # Jika tangan terdeteksi
        fingers = []

        # Logika untuk deteksi ibu jari (thumb)
        if lmList[tipIds[0]][1] > lmList[tipIds[0] - 1][1]: # Jika ibu
jari di sebelah kanan (horizontal)
            fingers.append(1) # Jari terbuka
        else:
            fingers.append(0) # Jari tertutup

        # Logika untuk 4 jari lainnya
        for id in range(1, 5): # Iterasi untuk telunjuk, jari tengah,
manis, dan kelingking
            if lmList[tipIds[id]][2] < lmList[tipIds[id] - 2][2]: #
Jika ujung jari lebih tinggi
                fingers.append(1) # Jari terbuka
            else:
                fingers.append(0) # Jari tertutup

        totalFingers = fingers.count(1) # Hitung jumlah jari yang
terbuka
        print(totalFingers) # Cetak jumlah jari yang terbuka ke
terminal

        # Mengirim data jumlah jari ke hardware melalui serial jika
waktu delay tercapai
        current_time = time.time()
        if current_time - last_sent_time >= delay_time:
            if tempSend != totalFingers: # Hanya kirim data jika jumlah
jari berubah
                ser.write(str(totalFingers).encode()) # Kirim jumlah
jari dalam bentuk string
                last_sent_time = current_time # Perbarui waktu
pengiriman terakhir
                tempSend = totalFingers # Simpan data terakhir yang
dikirim

        # Menampilkan gambar overlay sesuai dengan jumlah jari yang
terdeteksi
        h, w, c = overlayList[totalFingers - 1].shape
        img[0:h, 0:w] = overlayList[totalFingers - 1] # Overlay gambar
di pojok kiri atas frame

```

```

# Menghitung FPS (Frames Per Second)
cTime = time.time()
fps = 1 / (cTime - pTime)
pTime = cTime

# Menampilkan FPS pada frame (bisa diaktifkan jika perlu)
# cv2.putText(img, f'FPS: {int(fps)}', (400, 70),
cv2.FONT_HERSHEY_PLAIN, 3, (255, 0, 0), 3)

# Menampilkan frame video
cv2.imshow("Image", img)
cv2.waitKey(1) # Menunggu input keyboard selama 1ms (untuk looping
terus menerus)

```

Program pada ESP32 yang berada pada PC:

```

#include <esp_now.h> // Library untuk komunikasi ESP-NOW
#include <WiFi.h> // Library untuk koneksi WiFi (mode STA
digunakan di sini)
#include <esp_wifi.h> // Library tambahan untuk kontrol channel
WiFi
#include <Wire.h> // Library untuk komunikasi I2C
#include <LiquidCrystal_I2C.h> // Library untuk mengontrol LCD I2C

// Inisialisasi LCD I2C dengan alamat 0x27, 16 kolom, dan 2 baris
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Alamat MAC ESP32 penerima (target komunikasi ESP-NOW)
uint8_t espAddress[] = { 0x54, 0x43, 0xB2, 0xC3, 0x1C, 0x84 }; //
Gantilah dengan alamat MAC ESP32 Anda

// Struktur data untuk pesan yang akan dikirim melalui ESP-NOW
typedef struct struct_message {
    int sensorData; // Data sensor atau data yang akan dikirim
} struct_message;

// Variabel untuk menyimpan data yang akan dikirim
struct_message myData;

// Variabel untuk menunggu waktu delay pengiriman data
int waitingTime;

// Callback ketika data telah dikirim
void onDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
    Serial.print("\r\nLast Packet Send Status: ");
    // Tampilkan status pengiriman apakah berhasil atau gagal
    Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Delivery Success" :
"Delivery Fail");
}

void setup() {
    Serial.begin(115200); // Inisialisasi komunikasi serial dengan baud
rate 115200

```

```

// Inisialisasi dan setup LCD
lcd.begin(16, 2);
lcd.init();           // Inisialisasi LCD
lcd.backlight();      // Aktifkan lampu latar LCD
lcd.setCursor(0, 0);
lcd.print("Track Your Hand!"); // Tampilkan pesan awal di baris 1
lcd.setCursor(0, 1);
lcd.print("    Ready!    "); // Tampilkan pesan siap di baris 2

// Konfigurasi WiFi sebagai mode Station (STA)
WiFi.mode(WIFI_STA);

// Mengatur channel WiFi ke channel 1
esp_wifi_set_channel(1, WIFI_SECOND_CHAN_NONE);

// Inisialisasi ESP-NOW
if (esp_now_init() != ESP_OK) {
    Serial.println("Error initializing ESP-NOW");
    return;
}

// Daftarkan fungsi callback untuk notifikasi pengiriman data
esp_now_register_send_cb(onDataSent);

// Konfigurasi informasi peer (penerima data)
esp_now_peer_info_t peerInfo;
memcpy(peerInfo.peer_addr, espAddress, 6); // Salin alamat MAC ke
konfigurasi peer
peerInfo.channel = 0;           // Gunakan channel default
peerInfo.encrypt = false;      // Komunikasi tanpa enkripsi

// Tambahkan peer ke daftar ESP-NOW
if (esp_now_add_peer(&peerInfo) != ESP_OK) {
    Serial.println("Failed to add peer");
    return;
}
}

void loop() {
    // Mengecek jika ada data masuk melalui serial monitor
    if (Serial.available()) {
        String receivedData = Serial.readString(); // Baca data dari serial
        input
        int numRecv = receivedData.toInt();        // Konversi data string
        ke integer

        // Tampilkan data sensor pada LCD
        lcd.setCursor(0, 1);
        lcd.print("    "); // Bersihkan area tulisan
        lcd.setCursor(2, 1); // Atur posisi kursor di kolom 2, baris 2
        lcd.print(numRecv);  // Cetak data angka yang diterima
    }
}

```

```

        // Jika nilai kurang dari 6, kirimkan data menggunakan ESP-NOW
        if (numRecv < 6) {
            waitingTime = 0; // Tidak ada delay
            myData.sensorData = numRecv; // Isi struktur data dengan data
            sensor

            // Kirim data ke alamat MAC tujuan menggunakan ESP-NOW
            esp_err_t result = esp_now_send(espAddress, (uint8_t *)&myData,
            sizeof(myData));
            if (result == ESP_OK) {
                lcd.setCursor(7, 1);
                lcd.print("Success "); // Tampilkan status sukses di LCD
            } else {
                lcd.setCursor(7, 1);
                lcd.print("Failed "); // Tampilkan status gagal di LCD
            }
        }
        else {
            waitingTime = 300; // Jika nilai >= 6, tambahkan delay 300 ms
        }

        delay(waitingTime); // Delay sesuai nilai waitingTime
    }
}

```

Program pada ESP32 yang terhubung dengan beban(kipas angin):

```

#include <WiFi.h> // Library untuk konfigurasi WiFi
#include <esp_now.h> // Library untuk komunikasi ESP-NOW
#include <esp_wifi.h> // Library untuk konfigurasi channel WiFi
#include <Wire.h> // Library untuk komunikasi I2C
#include <LiquidCrystal_I2C.h> // Library untuk kontrol LCD I2C
#include <EEPROM.h> // Library untuk menyimpan data ke EEPROM

// Pin definisi untuk motor
#define IN1 14 // Pin digital untuk mengontrol arah motor
#define IN2 27 // Pin digital untuk mengontrol arah motor
#define ENA 26 // Pin PWM untuk mengontrol kecepatan motor

// Inisialisasi LCD I2C (alamat 0x27, 20 kolom, 4 baris)
LiquidCrystal_I2C lcd(0x27, 20, 4);

// Struktur data untuk komunikasi ESP-NOW
typedef struct struct_message {
    int recData; // Variabel untuk menyimpan data yang diterima
} struct_message;

// Variabel global untuk struktur data
struct_message myData;

// Variabel untuk menyimpan nilai terakhir dan alamat EEPROM
int savedValue, eepromAddress = 0;

```



```

// Callback ketika data diterima melalui ESP-NOW
void onDataRecv(const esp_now_recv_info *mac_addr, const uint8_t
*incomingData, int len) {
    memcpy(&myData, incomingData, sizeof(myData)); // Salin data yang
diterima ke `myData`
    Serial.println(myData.recData);                // Cetak data yang
diterima ke serial monitor

    // Kontrol motor dan tampilan LCD berdasarkan data yang diterima
    if (myData.recData == 1) {
        saveValueToEEPROM(1); // Simpan nilai ke EEPROM
        lcd.setCursor(0, 3);
        lcd.print("|    Very Low    |");
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        analogWrite(ENA, 159); // Atur kecepatan motor
        delay(100);
        analogWrite(ENA, 127);
    } else if (myData.recData == 2) {
        saveValueToEEPROM(2);
        lcd.setCursor(0, 3);
        lcd.print("|    Low    |");
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        analogWrite(ENA, 159);
    } else if (myData.recData == 3) {
        saveValueToEEPROM(3);
        lcd.setCursor(0, 3);
        lcd.print("|    Medium    |");
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        analogWrite(ENA, 191);
    } else if (myData.recData == 4) {
        saveValueToEEPROM(4);
        lcd.setCursor(0, 3);
        lcd.print("|    High    |");
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        analogWrite(ENA, 223);
    } else if (myData.recData == 5) {
        saveValueToEEPROM(5);
        lcd.setCursor(0, 3);
        lcd.print("|    Very High    |");
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        analogWrite(ENA, 255);
    } else if (myData.recData == 0) {
        saveValueToEEPROM(0);
        lcd.setCursor(0, 3);
        lcd.print("|    Stop    |");
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, LOW);
        analogWrite(ENA, 0);
    }
}

```

```

    }
}

// Fungsi `setup()` untuk konfigurasi awal
void setup() {
    Serial.begin(115200);    // Inisialisasi komunikasi serial
    EEPROM.begin(512);       // Inisialisasi EEPROM dengan ukuran 512
    byte

    // Atur pin motor sebagai output
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(ENA, OUTPUT);

    // Inisialisasi LCD I2C
    lcd.begin(20, 4);
    lcd.init();
    lcd.backlight(); // Aktifkan backlight pada LCD

    // Menampilkan pesan awal pada LCD
    lcd.setCursor(0, 0);
    lcd.print("|   Effortless   |");
    lcd.setCursor(0, 1);
    lcd.print("| Speed Controller |");
    lcd.setCursor(0, 2);
    lcd.print("|-----|");
    lcd.setCursor(0, 3);
    lcd.print("|       Stop       |");

    // Membaca nilai terakhir dari EEPROM
    savedValue = EEPROM.read(eepromAddress);
    if (savedValue == -1) savedValue = 0; // Default jika EEPROM kosong
    else {
        switch (savedValue) { // Mengatur motor sesuai nilai yang tersimpan
            case 0:
                lcd.setCursor(0, 3);
                lcd.print("|       Stop       |");
                digitalWrite(IN1, LOW);
                digitalWrite(IN2, LOW);
                analogWrite(ENA, 0);
                break;
            case 1:
                lcd.setCursor(0, 3);
                lcd.print("|   Very Low   |");
                digitalWrite(IN1, HIGH);
                digitalWrite(IN2, LOW);
                analogWrite(ENA, 159);
                delay(100);
                analogWrite(ENA, 127);
                break;
            case 2:
                lcd.setCursor(0, 3);
                lcd.print("|       Low     |");

```

```

        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        analogWrite(ENA, 159);
        break;
    case 3:
        lcd.setCursor(0, 3);
        lcd.print("|      Medium      |");
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        analogWrite(ENA, 191);
        break;
    case 4:
        lcd.setCursor(0, 3);
        lcd.print("|      High      |");
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        analogWrite(ENA, 223);
        break;
    case 5:
        lcd.setCursor(0, 3);
        lcd.print("|    Very High    |");
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        analogWrite(ENA, 255);
        break;
    }
}

// Inisialisasi mode WiFi untuk ESP-NOW
WiFi.mode(WIFI_STA);          // Atur mode WiFi sebagai Station
WiFi.disconnect();            // Putuskan koneksi WiFi jika ada
esp_wifi_set_channel(1, WIFI_SECOND_CHAN_NONE); // Atur channel WiFi
ke 1

// Inisialisasi ESP-NOW
if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW Init Failed");
    return;
}

// Daftarkan callback untuk menerima data
esp_now_register_recv_cb(onDataRecv);
}

// Fungsi loop kosong (program berjalan berdasarkan callback)
void loop() {}

// Fungsi untuk menyimpan nilai ke EEPROM
void saveValueToEEPROM(int x) {
    EEPROM.write(eepromAddress, x); // Menulis nilai ke EEPROM
    EEPROM.commit();                 // Commit data ke EEPROM agar
tersimpan permanen
}

```